

DATA-260, Homework 1

Pinal Pawar

Section 0, Personal Configuration and Domain

Value	Definition	This submission
SID4	Last 4 digits of SJSU Student ID	7875
PORT_BASE	$8000 + (\text{SID4} \bmod 900)$	8675
PREFIX	"s" + SID4	s7875
SEED	SID4	7875
VERIFY_SEED	$260000 + \text{SID4}$	267875
DOMAIN_ID	$\text{SID4} \bmod 8$	3 (Grocery supply and recall notices)

Hardware	MacBook Air, Apple M2, 16GB RAM
Local model	qwen3:8b (served via Ollama)
Tagged commit	see the hw1 tag on <a href="https://github.com/<user>/data260-7875">github.com/<user>/data260-7875

HTML Point 3, DOMAIN_SCHEMA.md

Entity: GroceryRecallNotice

Field	Type	Required	Description
productName	text (primary field)	Yes	Name of the recalled grocery product, including size/variant (e.g. "Organic Baby Spinach, 5oz bag")
manufacturer	text (secondary field)	Yes	Company or brand that manufactured/issued the recall (e.g. "FreshFields Farms")
submitterEmail	email	Yes	Email address of the person submitting the recall report
description	textarea	Yes	Details of the recall: reason, affected lot/batch numbers, symptoms or risks reported
category	dropdown (enum)	Yes	Type of recall issue, one of: Supply Shortage, Bacterial Contamination, Foreign Material, Mislabeling / Packaging Defect
agreeToTerms	checkbox (boolean)	Yes	Submitter confirms agreement to terms and conditions

Category values: 1) Bacterial Contamination (e.g. E. coli, Salmonella, Listeria) 2) Undeclared Allergen 3) Foreign Material Contamination 4) Mislabeling / Packaging Defect 5) Supply Shortage

Part 1, HTML / JavaScript / Deployment

HTML form

The page title is **HW1-Pinal**, and I used an `<h1>` for the domain heading since that's the largest heading tag and that's what the assignment wanted. The Product field has autofocus on it, so the cursor is already sitting there when the page loads (you can see the blue outline in the screenshot below). All four of the text/email/textarea fields are required, the category dropdown has four options that fit the domain plus a disabled placeholder, and there's a checkbox for "I agree to the terms and conditions" that's required too.

```
<input type="text" id="product" placeholder="Enter the product" required autofocus>
...
<textarea id="description" placeholder="Describe the issue" required></textarea>
...
<select id="category">
  <option value="" disabled selected>Select category</option>
  <option value="supplyShortage">Supply Shortage</option>
  <option value="bacterialContamination">Bacterial Contamination</option>
  <option value="foreignMaterial">Foreign Material</option>
  <option value="Mislabelling">Mislabeling / Packaging Defect</option>
</select>
...
<input type="checkbox" id="agreeTerms">
<label for="agreeTerms">I agree to the terms and conditions</label>
```

Grocery supply and recall notices

Product:

Manufacturer:

Email:

Description:

Category: ▼

☐ I agree to the terms and conditions

Output, form on page load, Product field auto-focused.

Grocery supply and recall notices

Product:

Manufacturer:

Email:

Description:

Category:

✓ Select category

Supply Shortage

Bacterial Contamination

Foreign Material

Mislabeling / Packaging Defect

☐ I agree

Output, category dropdown open, showing all four options.

JavaScript validation

The validation is an arrow function that checks two things before it lets the form submit: description longer than 25 characters, and the terms checkbox checked. If either one fails, it pops an alert telling you specifically what's wrong instead of just doing nothing.

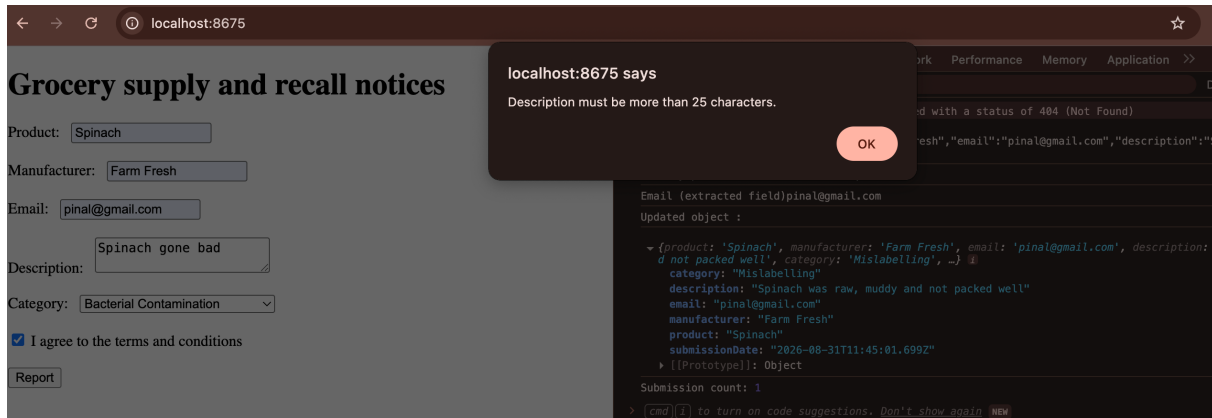
```
const validateForm = () => {
  const description = document.getElementById("description").value
  const agreed = document.getElementById("agreeTerms").checked

  if (description.trim().length <= 25) {
    alert("Description must be more than 25 characters.")
    return false
  }
  if (!agreed) {
    alert("You must agree to the terms and conditions.")
    return false
  }
}
```

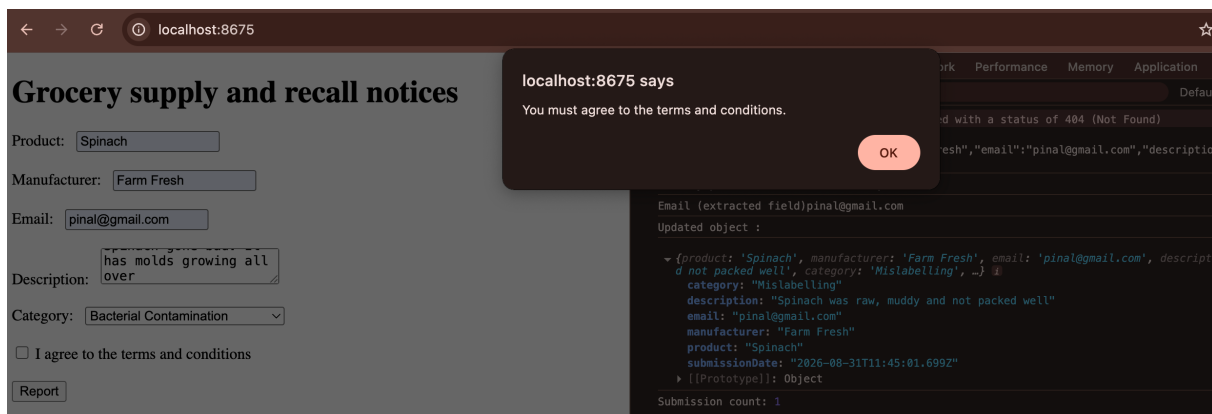
```

    }
    return true
  }
}

```



Output, validation failure, description under 25 characters.



Output, validation failure, terms checkbox not checked.

JSON, destructuring, spread, closure

When the form submits successfully, the form data gets turned into a JSON string and logged, then parsed back and destructured to pull out just the product and email fields. The spread operator makes a copy of that parsed object and adds a new submissionDate field on top. There's also a closure (createCounter) that keeps its own private counter running across submissions.

```

const createCounter = () => {
  let count = 0
  return () => ++count
}
const submissionCount = createCounter()

const jsonFormData = JSON.stringify(formData)
console.log("Form data to JSON string : \n" + jsonFormData)

const parsedFormData = JSON.parse(jsonFormData)
const {product: productfromData, email: emailfromData} = parsedFormData
console.log("Primary product (extracted field):" + productfromData)
console.log("Email (extracted field)" + emailfromData)

const updateFormData = {...parsedFormData, submissionDate: new Date().toISOString()}

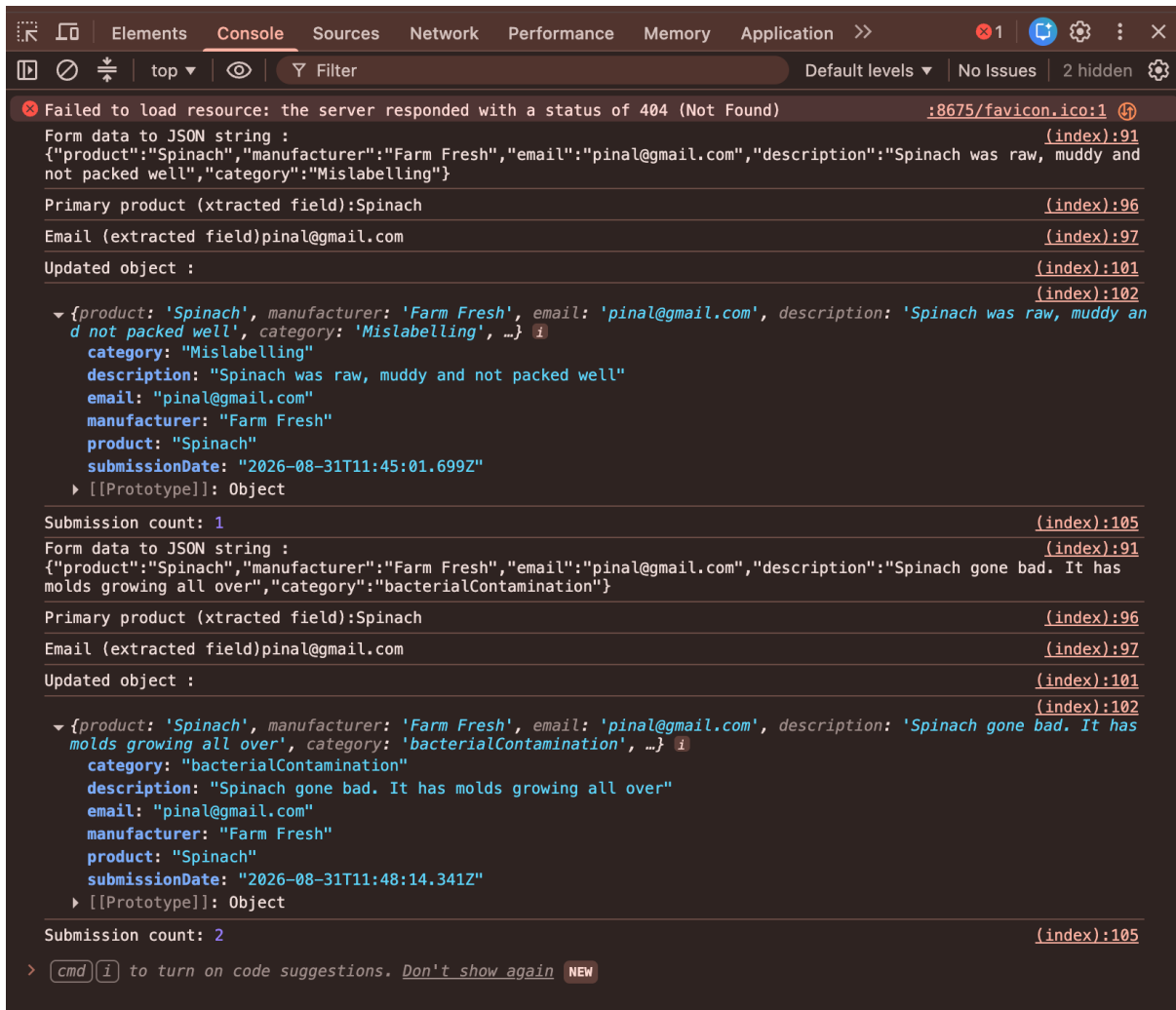
```

```

console.log("Updated object : ")
console.log(updateFormData)

console.log("Submission count:", submissionCount())

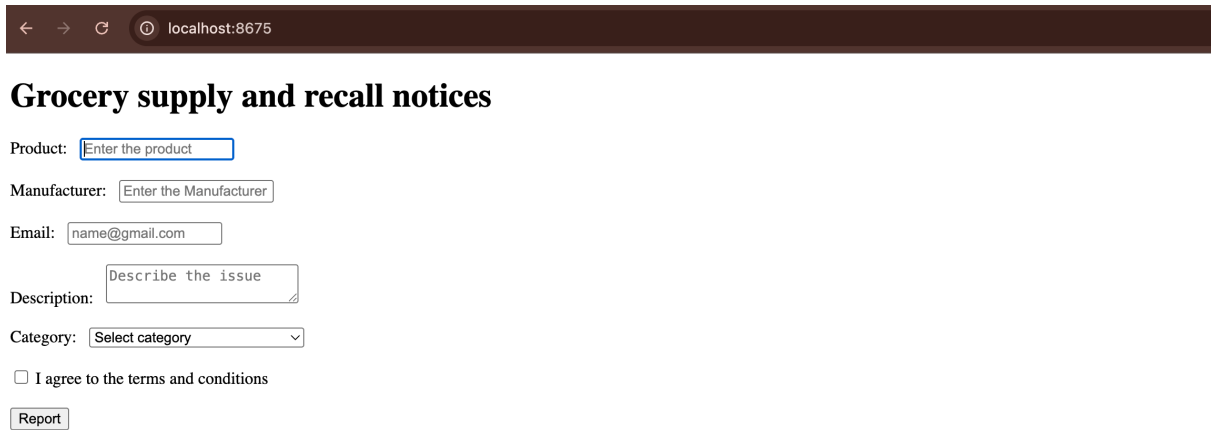
```



Output, console across two submissions: JSON string, destructured fields, spread-updated object, and the closure-based submission counter going from 1 to 2.

Deployment, Docker (localhost)

I packaged the app with an nginx-based Dockerfile, built it locally, and ran it with `docker run -d -p 8675:8675 my-web-app`. That makes it reachable at `http://localhost:8675`, which matches my `PORT_BASE`.



The screenshot shows a web browser window with the address bar displaying 'localhost:8675'. The page title is 'Grocery supply and recall notices'. The form contains the following fields and elements:

- Product:** A text input field with the placeholder text 'Enter the product'.
- Manufacturer:** A text input field with the placeholder text 'Enter the Manufacturer'.
- Email:** A text input field with the placeholder text 'name@gmail.com'.
- Description:** A text area with the placeholder text 'Describe the issue'.
- Category:** A dropdown menu with the text 'Select category'.
- ☐ I agree to the terms and conditions
- Report** button

The app running locally via Docker at localhost:8675.

Deployment, AWS ECS (Fargate)

I pushed that same image to ECR and deployed it as an ECS Fargate Service (Linux/ARM64, 0.25 vCPU / 0.5GB, one task) with a security group that allows inbound TCP 8675 and a public IP assigned to it.

The app running on AWS, reachable at its public IP on port 8675.

ECS Service overview: Active, 1/1 tasks running, deployment status Success.

Part 2, Agentic AI (Planner → Reviewer → Finalizer)

1. Command used

```
python agents_demo.py --title "Organic Baby Spinach Recall" --content "Routine testing detected Listeria in lot #L4471. Affected bags were sold in California and Nevada stores."
```

2. Console output: Planner, Reviewer, Finalized, Publish package

```
(data260) atharvamundada@Atharvas-MacBook-Air Downloads % ollama list
NAME      ID              SIZE    MODIFIED
qwen3:8b  580a1f067a9f  5.2 GB  2 hours ago
(data260) atharvamundada@Atharvas-MacBook-Air Downloads % python agents_demo.py --title "Organic Baby Spinach Recall" --content "Routine testing detected Listeria in lot #L4471. Affected bags were sold in California and Nevada stores."

--- Planner (412273 ms) ---
{
  "thought": "The blog post details a Listeria contamination in organic baby spinach, leading to a recall in specific states.",
  "message": "The blog post discusses a Listeria contamination in organic baby spinach, prompting a recall in California and Nevada.",
  "data": {
    "tags": [
      "Food Safety Recall",
      "Organic Produce Recall",
      "Listeria Contamination Recall"
    ],
    "summary": "Routine testing found Listeria in organic baby spinach bags sold in California and Nevada, prompting a recall.",
    "issues": []
  }
}

--- Reviewer (117654 ms) ---
{
  "thought": "Tags focus on specific recall reasons and locations; summary concisely captures contamination and affected areas.",
  "message": "Blog details a Listeria-contaminated organic spinach recall impacting California and Nevada stores due to routine testing findings.",
  "data": {
    "tags": [
      "Listeria Outbreak",
      "Organic Produce Recall",
      "Food Safety Alert"
    ],
    "summary": "Organic baby spinach recalled for Listeria contamination in California and Nevada stores.",
    "issues": []
  }
}

Finalized Output
{
  "thought": "Prioritizing food safety, recall specifics, and contamination type for tags. Summary focuses on key details from the blog.",
  "message": "Blog reports Listeria contamination in organic spinach leading to a recall in California and Nevada stores.",
  "data": {
    "tags": [
      "Food Safety",
      "Recall",
      "Listeria Outbreak"
    ],
    "summary": "Organic baby spinach recalled due to Listeria contamination found in California and Nevada stores.",
    "issues": []
  }
}

Publish Package
{
  "title": "Organic Baby Spinach Recall",
  "email": "student@example.com",
  "content": "Routine testing detected Listeria in lot #L4471. Affected bags were sold in California and Nevada stores.",
  "agents": {
    "transcript": [
      {
        "role": "Planner",
        "content": "The blog post discusses a Listeria contamination in organic baby spinach, prompting a recall in California and Nevada."
      },
      {
        "role": "Reviewer",
        "content": "Blog details a Listeria-contaminated organic spinach recall impacting California and Nevada stores due to routine testing findings."
      }
    ],
    "final": {
      "tags": [
        "Food Safety",
        "Recall",
        "Listeria Outbreak"
      ],
      "summary": "Organic baby spinach recalled due to Listeria contamination found in California and Nevada stores.",
      "issues": []
    }
  },
  "submissionDate": "2026-08-31T04:57:04Z"
}
```

Ollama model in use (qwen3:8b), the command, and Planner / Reviewer / Finalized Output.

```
Publish Package
{
  "title": "Organic Baby Spinach Recall",
  "email": "student@example.com",
  "content": "Routine testing detected Listeria in lot #L4471. Affected bags were sold in California and Nevada stores.",
  "agents": {
    "transcript": [
      {
        "role": "Planner",
        "content": "The blog post discusses a Listeria contamination in organic baby spinach, prompting a recall in California and Nevada."
      },
      {
        "role": "Reviewer",
        "content": "Blog details a Listeria-contaminated organic spinach recall impacting California and Nevada stores due to routine testing findings."
      }
    ],
    "final": {
      "tags": [
        "Food Safety",
        "Recall",
        "Listeria Outbreak"
      ],
      "summary": "Organic baby spinach recalled due to Listeria contamination found in California and Nevada stores.",
      "issues": []
    }
  },
  "submissionDate": "2026-08-31T04:57:04Z"
}
```

Full Publish package printed to console.

3. Short answers

Q1. The 3 final tags obtained: Food Safety, Recall, Listeria Outbreak

Q2. The final summary (≤25 words): "Organic baby spinach recalled due to Listeria contamination found in California and Nevada stores."

Q3. Did the Reviewer agent change anything? Yes. The Planner's first attempt gave three tags that were pretty much the same idea said three different ways, all just variations on "recall." The Reviewer is instructed to catch generic or repetitive tags, and that's what happened here: it swapped two of them out for Listeria Outbreak and Food Safety Alert so they actually covered different angles instead of repeating the same word. Then the Finalizer cleaned that up one more step into Food Safety, Recall, Listeria Outbreak.

4. Explanation of each step, in my own words

Planner is the first agent that touches the input. It just reads the title and content cold, with nothing from the other agents yet, and takes a first shot at 3 tags plus a short summary.

Reviewer comes next. It gets the Planner's attempt and checks it over, whether the tags are too vague or repetitive, whether the summary ran long, and rewrites whatever it decides needs fixing.

Finalizer is the last one. It sees both what the Planner came up with and what the Reviewer changed, and picks and cleans up the version that actually gets printed as the result.

All three are actually the same SimpleAgent class under the hood, they just get handed a different one-line system prompt inside main(). Each one builds its prompt from that instruction plus a shared template with the task and whatever transcript exists so far, so Planner starts from nothing, Reviewer sees what Planner said, and Finalizer sees both. That's really the entire "agents talking to each other" trick, each one just reads what came before it. One thing I noticed while testing this: whatever the model spits out gets forced into the required shape afterward by parse_and_coerce(), trimming the summary to 25 words and capping tags at exactly 3, so the final output matches spec even if the model itself wanders off script.

Part 3, Measuring Non-Determinism

Fixed input (saved once, unchanged, as reports/hw01/cases/nondeterminism_input.json): a title and content about a peanut butter recall over salmonella contamination. The pipeline was run 20 times at temperature 0.7 and 20 times at temperature 0.0 against that exact input (40 runs total).

```
python run_nondeterminism.py
```

```
ter jars tested positive for salmonella. Several people who ate it reported getting sick with stomach cramps and fever. --temperature 0.0
[2026-08-31 03:37:03] END   runs=40 temp=0.0 latency_ms=350187 tags=['Peanut Butter Recall', 'Salmonella Contamination', 'Consumer Illnesses']
[2026-08-31 03:37:03] All runs complete. Computing stats...
[2026-08-31 03:37:03] Wrote reports/hw01/METRICS.md
[2026-08-31 03:37:03] Wrote reports/hw01/raw/nondeterminism_raw.json

Done. See reports/hw01/METRICS.md and reports/hw01/raw/nondeterminism_raw.json
(data260) atharvamundada@Atharvas-MacBook-Air Downloads % cat reports/hw01/METRICS.md
# Non-Determinism Metrics (Part 3)

Fixed input: 'reports/hw01/cases/nondeterminism_input.json'

| Metric | Temp 0.7 | Temp 0.0 |
|---|---|---|
| Distinct tag sets | 15 | 2 |
| Tags in all 20 runs | (none) | Peanut Butter Recall, Salmonella Contamination |
| Tags in exactly 1 run | 9 tags (e.g. Consumer Health Alert, Food Recall, Public Health Alert) | (none) |

| Metric | Temp 0.7 | Temp 0.0 |
|---|---|---|
| Latency p50 (ms) | 455488 | 311498 |
| Latency p95 (ms) | 753363 | 371239 |
| Latency p99 (ms) | 764860 | 378558 |
(data260) atharvamundada@Atharvas-MacBook-Air Downloads %
```

Output, run completion and the auto-generated METRICS.md, printed to console.

Metric	Temp 0.7	Temp 0.0
Distinct tag sets	15	2
Tags in all 20 runs	(none)	Peanut Butter Recall, Salmonella Contamination
Tags in exactly 1 run	9 tags (e.g. Consumer Health Alert, Food Recall, Public Health Alert)	(none)

Metric	Temp 0.7	Temp 0.0
Latency p50 (ms)	455,488 (~7.6 min)	311,498 (~5.2 min)
Latency p95 (ms)	753,363 (~12.6 min)	371,239 (~6.2 min)
Latency p99 (ms)	764,860 (~12.7 min)	378,558 (~6.3 min)

Explanation

The difference between the two temperatures was pretty noticeable. At 0.7, running the exact same input 20 times gave me 15 different tag combinations. Almost every run came back with something different, and nothing showed up in all 20 runs. Nine tags only appeared once, including near-duplicate phrasings the model seemed to make up on the spot: "Consumer Health Alert" vs "Consumer Health Alerts" vs "Consumer Health Risks," which are all really the same idea worded slightly differently. So if two people submitted the exact same recall report at temperature 0.7, they'd probably get two different sets of tags back.

At 0.0 it was the opposite. Only 2 unique tag combinations came out of all 20 runs, and two tags, Peanut Butter Recall and Salmonella Contamination, showed up in every single run with no one-off tags anywhere. Two people running that same input at temperature 0.0 would almost certainly land on the same result.

Latency followed the same trend. At 0.7, runs took anywhere from about 7.6 minutes (median) up to 12.7 minutes (p99), so a pretty wide spread. At 0.0 things were both faster (median around 5.2 minutes) and a lot more consistent (p99 only about 6.3 minutes).

Where this kind of variation is fine: something like the tags shown under a blog post about a recall. Seeing "Consumer Health Alert" one time and "Public Health Alert" the next isn't really an issue, since both are reasonable and say roughly the same thing.

Where it's not fine: if this same pipeline were deciding whether a specific product batch gets officially flagged for recall. A decision like that needs to come out the same way every time no matter who runs it, so temperature 0.0, or no randomness at all, is the right call there, not 0.7.

Part 4, Model Client and Token Accounting

All model calls go through a single adapter, `src/model_client.py`, which exposes `complete(messages, tools=None)`. `hw1_client.py` is a small CLI built on top of it, with an `AGENT.md` system prompt instructing strict bullet-only code review. A five-turn conversation was run, printing per-turn token counts after every response and cumulative totals via `/stats` after turns 3 and 5, and again on `/exit`.

```
python hw1_client.py
```

```
python hw1_client.py
hw1_client -- type a message, or /stats, or /exit

[you> Review this code: def add(a,b): return a+b
assistant> - The function 'add' correctly returns the sum of 'a' and 'b'.
- No syntax errors are present in the code.
- The code is concise and achieves its intended purpose.
- Consider adding type hints for better clarity and maintainability.
- Ensure the function is used in a context where integer or float inputs are expected.
[turn 1] input_tokens=147 output_tokens=747 total_tokens=894

[you> Review this code: def divide(a, b): return a / b
assistant> - The function 'divide' lacks error handling for division by zero, which could raise a 'ZeroDivisionError'.
- The code contains an unintended '/think' suffix in the return statement, which is likely a typo.
- Type hints are missing, reducing clarity about expected input types.
- The division operator '/' performs floating-point division; consider using '//' for integer division if intended.
- Ensure the function is used with valid non-zero denominators to avoid runtime errors.
[turn 2] input_tokens=243 output_tokens=597 total_tokens=840

[you> Review this code: for i in range(10): print(i)
assistant> - The '/think' suffix at the end of the code is invalid syntax and should be removed.
- The loop correctly iterates from 0 to 9 and prints each value.
- Consider adding a comment to clarify the purpose of the loop.
- The code could be made more concise by using a list comprehension or generator if the output is needed for further processing.
- Ensure the loop is used in a context where printing numbers sequentially is intentional.
[turn 3] input_tokens=368 output_tokens=341 total_tokens=709

[you> /stats
[stats] turns=3 cumulative_input_tokens=758 cumulative_output_tokens=1685 cumulative_total_tokens=2443 history_length_chars=2254

[you> Review this code: x = [1, 2, 3]; y = x; y.append(4)
assistant> - The '/think' suffix at the end of the code is invalid syntax and should be removed.
- 'y' is assigned to reference the same list as 'x', so modifications to 'y' (like 'append(4)') will also modify 'x'.
- The code does not output the modified list, so the changes are not visible unless printed.
- Consider using 'y = x.copy()' or 'y = list(x)' to create a separate copy if unintended mutation is a concern.
- The code is concise but could benefit from a comment explaining the reference behavior.
[turn 4] input_tokens=496 output_tokens=442 total_tokens=938

[you> Review this code: class Foo: pass
assistant> - The '/think' suffix at the end of the code is invalid syntax and should be removed.
- The class 'Foo' is defined but contains no attributes or methods, making it functionally empty.
- The semicolon after 'class Foo:' is unnecessary and should be omitted.
- Consider adding methods or attributes to the class if it is intended to represent a concrete concept.
- The class name 'Foo' is valid but could be more descriptive depending on its purpose.
[turn 5] input_tokens=636 output_tokens=337 total_tokens=973

[you> /stats
[stats] turns=5 cumulative_input_tokens=1890 cumulative_output_tokens=2464 cumulative_total_tokens=4354 history_length_chars=3450

[you> /exit

[exit] total_turns=5 cumulative_input_tokens=1890 cumulative_output_tokens=2464 cumulative_total_tokens=4354
(data260) atharvamundada@Atharvas-MacBook-Air Downloads %
```

Output, five-turn conversation: per-turn token counts, /stats after turn 3 and after turn 5, and the final cumulative totals on /exit.

Conceptual answers

(i) Why is prior conversation context resent with every turn? The model has no memory between calls on its own, so the client has to resend the whole conversation so far every single time, or it would have no idea what was said earlier. This shows up directly in the numbers: input tokens went 147, then 243, 368, 496, 636 across the five turns, climbing every time even though each new message I typed was short. That climb is just the growing history getting resent each turn, not longer messages on my end.

(ii) How is a system prompt different from a user message? The system prompt, the bullet-only instruction from `AGENT.md` in this case, gets set once and holds for the entire conversation. That's why all 5 responses came back as strictly bullet points. A user message is just whatever got typed in one specific turn, five separate one-off messages here.

(iii) Why do input tokens grow over a conversation? Same root cause as (i). The full history gets resent every turn, and that history keeps getting longer, so input tokens climb right along with it. By turn 5 it had

gone from 147 up to 636 tokens, over 4x, and none of that came from longer messages, just accumulated history.

(iv) What eventually limits that growth? Every model has a context window, a hard limit on tokens it can hold at once. So this can't keep going forever. Once a conversation gets close to that ceiling, older messages have to get dropped, summarized, or truncated to keep it going.